LAPORAN TUGAS BESAR 1 IF3170
INTELEGENSI ARTIFISIAL

OPTIMASI PENGISIAN PAKET PETI KEMAS MENGGUNAKAN ALGORITMA LOCAL SEARCH

Nama Kelompok: IF3170_G41_OpenBobi

Anggota Kelompok:

13524027 - Aufa Rienaldifaza Ahmad

13524071 - Kalyca Nathania Benedicta Manullang

13524077 - Yavie Azka Putra Araly

Dosen Pengampu:

Dr. Nur Ulfa Maulidevi, S.T, M.Sc.

Dr. Fariska Zakhralativa Ruskanda, S.T., M.T

Dr. Edward Ferdian S.T, B.Eng, M.Sc.

DAFTAR ISI

BAB I **3**
DESKRIPSI PERSOALAN **3**
1.1 Latar Belakang & Deskripsi Permasalahan 3
1.2 Formulasi Permasalahan 4
Entitas & Atribut Permasalahan 4
Constraints (Batasan) 5
Objective Function 6
BAB II **7**
PEMBAHASAN DAN IMPLEMENTASI ALGORITMA **7**
2.1 Representasi Solusi dan State 7
2.2 Fungsi Objektif & Penanganan Constraint 8
2.3 Implementasi Algoritma Local Search 9
2.3.1 Hill-Climbing 9
2.3.2 Simulated Annealing 10
2.3.3 Genetic Algorithm 11
BAB III **16**
HASIL DAN ANALISIS EKSPERIMEN **16**
3.1 Setup Eksperimen 16
3.2 Hasil dan Analisis Hill Climbing 20
3.3 Hasil dan Analisis Simulated Annealing 21
3.4 Hasil dan Analisis Genetic Algorithm 22
3.5 Analisis Perbandingan dan Evaluasi 25
BAB IV **27**
KESIMPULAN DAN SARAN **27**
4.1 Kesimpulan 27
4.2 Saran 28
PEMBAGIAN TUGAS KELOMPOK **29**
REFERENSI **30**
LAMPIRAN **31**
Lampiran 1: Form Penggunaan Generative AI 31
Lampiran 2: Tangkapan Layar & Visualisasi Program 31

BAB I
DESKRIPSI PERSOALAN

1.1 Latar Belakang & Deskripsi Permasalahan

Dalam industri ekspedisi dan logistik darat, efisiensi pemanfaatan ruang penyimpanan serta kapasitas muatan pada armada transportasi merupakan faktor krusial yang menentukan profitabilitas operasional. Sebagai studi kasus pada Tugas Besar I kuliah IF3170 Inteligensi Artifisial ini, dikisahkan bahwa Keluarga Mejiro sedang melakukan ekspansi bisnis ke sektor logistik darat berskala kecil dengan mendirikan layanan pengiriman Mejiro Express yang mengusung slogan "Lebih kilat dari Umamusume". Dalam operasional uji coba ini, Special Week ditunjuk oleh Mejiro McQueen sebagai pengemudi truk. Namun, keterbatasan waktu yang dimiliki oleh Special Week akibat jadwal latihan menuntut adanya sistem komputasi cerdas yang mampu menyusun konfigurasi penempatan barang secara cepat dan optimal tanpa harus dilakukan secara manual.

Secara komputasional, persoalan yang dihadapi oleh Mejiro Express merupakan variasi dari permasalahan optimasi kombinatorial Three-Dimensional Single Bin Packing Problem (3D-BPP) atau Three-Dimensional Knapsack Problem (Martello, Pisinger, & Vigo, 1997). Sistem dihadapkan pada sekumpulan paket (package) berbentuk kubus atau balok dengan dimensi, bobot, nilai harga (value), waktu perkiraan tiba (Estimated Time of Arrival / ETA), serta karakteristik ketahanan barang (fragile / pecah-belah) yang beragam. Sebagaimana diformulasikan oleh Martello dkk. (1997) pada persoalan filling a single bin, sebagian subset paket harus dipilih dan dipetakan titik koordinat sudut kiri-bawah-belakangnya (x_i, y_i, z_i) ke dalam sebuah peti kemas truk (truck container) berdimensi (W, L, H) dan berkapasitas beban maksimum (MaxCapacity) tanpa menembus batas dinding maupun saling tumpang tindih (overlapping).Mengingat permasalahan 3D-BPP terbukti bersifat strongly NP-hard dan sangat sulit diselesaikan secara eksak dalam praktik nyata (Martello, Pisinger, & Vigo, 1997), penambahan enam kemungkinan orientasi rotasi paket, batasan gravitasi (support alas), serta batasan barang pecah-belah (fragile) semakin memperbesar ruang keadaan (state space) secara eksponensial. Pendekatan pencarian eksak seperti Branch-and-Bound membutuhkan waktu komputasi yang sangat besar seiring bertambahnya jumlah paket. Oleh karena itu, diperlukan pendekatan menggunakan algoritma Local Search yang mencakup varian algoritma Hill-Climbing, Simulated Annealing, dan Genetic Algorithm untuk mengeksplorasi ruang keadaan secara efisien dan menemukan

konfigurasi pemetaan paket (final state) yang memaksimalkan total value paket di dalam peti kemas dengan tetap mematuhi seluruh batasan fisik dan kapasitas muatan truk.

Sebagai bentuk pengembangan / bonus, program ini juga mendukung penanganan lebih dari satu peti kemas / Multi-Truk dengan mendistribusikan paket ke seluruh armada yang tersedia untuk memaksimalkan total value secara keseluruhan.

1.2 Formulasi Permasalahan

Entitas & Atribut Permasalahan

Berdasarkan spesifikasi tugas besar, entitas yang terlibat dalam permasalahan ini adalah sebagai berikut:

1. **Package (Paket)**

   Merepresentasikan barang yang akan dikirim.

Atribut	Tipe Data	Keterangan
id	String	Identifier unik untuk setiap paket.
dimensions	Tuple (int, int, int)	Ukuran paket berupa (w, l, h) (width, length, height).
orientation	Tuple (int, int, int)	Orientasi paket setelah rotasi 90° pada sumbu x, y, atau z.
value	Integer	Nilai harga dari paket (satuan unit).
weight	Integer	Berat dari paket (satuan unit berat).
is_fragile	Boolean	Menandakan apakah paket merupakan barang pecah-belah.
eta	Integer	*Estimated Time of Arrival* (perkiraan waktu kedatangan dalam hari).

2. **Truck (Peti Kemas Truk)**

   Merepresentasikan ruang muat tempat paket diletakkan.

Atribut	Tipe Data	Keterangan
label	String	Identifier unik untuk peti kemas.
dimensions	Tuple (int, int, int)	Ukuran peti kemas berupa (W, L, H) (width, length, height)
max_capacity	Integer	Kapasitas angkut maksimum (dalam satuan unit berat).

Constraints (Batasan)

Sebuah konfigurasi (state) dinyatakan valid jika dan hanya jika memenuhi seluruh batasan berikut:

1. **Batas Peti Kemas (Boundary)**

   Seluruh bagian paket harus berada di dalam ruang peti kemas. Untuk setiap paket yang diletakkan pada koordinat (x_i, y_i, z_i) dengan dimensi (w_i, l_i, h_i) harus memenuhi:

$$0 \leq y_i \text{ dan } y_i + l_i \leq L$$
$$0 \leq z_i \text{ dan } z_i + h_i \leq H$$

   Catatan: Paket juga diperbolehkan untuk berada sepenuhnya di luar peti kemas.

2. **Non-Overlapping (Tidak Tumpang Tindih)**

   Tidak boleh ada dua paket atau lebih yang menempati ruang tiga dimensi yang sama.

3. **Gravitasi / Support Alas (Tidak Melayang)**

   Paket tidak boleh melayang di udara. Suatu paket dikatakan memiliki support jika $z_i = 0$ (menyentuh alas truk) atau bagian alasnya ditopang secara penuh/parsial oleh bagian atas paket lain yang berada tepat di bawahnya.

4. **Aturan Barang Pecah-Belah (Fragile)**

Jika is_fragile bernilai true, maka paket tersebut tidak boleh diletakkan di bawah paket lain dan paket tersebut tidak boleh dijadikan sebagai penopang (support) bagi paket lain.

5. **Kapasitas Berat (Max Capacity)**

   Total berat seluruh paket yang diletakkan di dalam peti kemas tidak boleh melebihi max_capacity dari truk tersebut.

6. **Koordinat dan Orientasi**

   Koordinat (x,y,z) harus berupa bilangan bulat. Dimensi paket dapat dirotasi 90° terhadap sumbu x, y, dan z (mem bentuk 6 kemungkinan orientasi) selama tidak mengubah dimensi aslinya.

Objective Function

Tujuan utama dari optimasi ini adalah untuk memaksimalkan total value dari seluruh paket yang berhasil dimuat secara valid di dalam peti kemas truk (dan truk-truk lainnya jika menggunakan fitur Multi-Truk).

Secara matematis, fungsi objektif dirumuskan sebagai berikut:

$$\text{Maximize} \sum_{i \in P_{\text{placed}}} Value_i$$

Dimana Pplaced adalah himpunan paket yang berhasil diletakkan di dalam peti kemas dan tidak melanggar seluruh batasan (constraints) yang telah didefinisikan.

BAB II
PEMBAHASAN DAN IMPLEMENTASI ALGORITMA

2.1 Representasi Solusi dan State

Pada algoritma Hill-Climbing dan Simulated Annealing, solusi direpresentasikan secara langsung sebagai konfigurasi penempatan paket di dalam peti kemas (kelas State). Sebuah state memuat pemetaan placed dari label paket ke objek Placement, yaitu pasangan posisi sudut kiri-bawah-belakang (x, y, z) dan dimensi hasil rotasi (w, d, h). Paket yang labelnya tidak terdapat pada placed berada di luar truk dan dicatat pada daftar outside. Dengan demikian, sebuah state menentukan paket mana yang dimuat, koordinat penempatannya, dan orientasinya.

Setiap state yang dibangkitkan oleh kelas StateSpace dijamin feasible, karena setiap penempatan paket diperiksa oleh fungsi _can_add sebelum dimasukkan ke dalam state. Oleh sebab itu, tidak ada state yang melanggar batasan dan tidak diperlukan fungsi penalti. Hill-Climbing dan Simulated Annealing berbagi ruang keadaan yang sama, sedangkan Genetic Algorithm menggunakan representasi tak langsung yang dijelaskan pada subbab 2.3.3.

Initial state dibangkitkan secara acak oleh generate_initial_state. Urutan label paket diacak, lalu setiap paket diberi satu rotasi acak dari daftar rotasi unik hasil Box.get_rotations(). Titik kandidat penempatan dibangkitkan oleh fungsi _points, yaitu titik asal (0, 0, 0) ditambah tiga titik proyeksi sudut dari setiap paket yang sudah terpasang, yaitu (x + w, y, z), (x, y + d, z), dan (x, y, z + h). Titik yang berada di luar dimensi peti kemas dibuang. Urutan titik kandidat diacak, dan paket diletakkan pada titik pertama yang lolos _can_add. Jika tidak ada titik yang lolos, paket tersebut tetap berada di luar truk.

Ruang keadaan dijelajahi melalui empat jenis langkah suksesor:

- ADD: memasukkan satu paket dari luar truk ke satu titik kandidat dengan salah satu rotasinya.
- REMOVE: mengeluarkan satu paket yang sudah terpasang dari truk. Langkah ini hanya diizinkan apabila seluruh paket yang tersisa tetap tertopang (fungsi

_all_supported), sehingga tidak ada paket yang melayang setelah penopangnya dikeluarkan.

- MOVE: mengeluarkan satu paket terpasang lalu menempatkannya kembali pada titik atau rotasi yang berbeda.
- SWAP: mengeluarkan satu paket terpasang dan memasukkan satu paket lain dari luar truk pada titik kandidat mana pun.

Fungsi neighbors membangkitkan seluruh suksesor feasible dan digunakan oleh Hill-Climbing. Fungsi ini membentuk satu basis untuk state saat ini dan satu basis untuk setiap kemungkinan pengeluaran satu paket, kemudian mencoba setiap paket di luar basis pada setiap rotasi dan titik kandidat. Penempatan ulang paket yang sama pada posisi dan rotasi yang identik dikecualikan. Fungsi random_neighbor, yang digunakan oleh Simulated Annealing, memilih satu jenis langkah secara seragam dan mencoba hingga 50 kali (max_tries) untuk mendapatkan satu suksesor feasible. Jika semua percobaan gagal, fungsi ini mengembalikan None.

2.2 Fungsi Objektif & Penanganan Constraint

Fungsi objektif dihitung oleh evaluate_fitness pada modul objective.py dan dipilih melalui parameter objective_type. Tipe yang tidak dikenal dialihkan ke mode default. Seluruh algoritma memaksimalkan nilai f. Tiga mode yang tersedia adalah:

- default: $f(S) = \Sigma\ v_i$ untuk seluruh paket i yang berada di dalam truk, yaitu total value paket yang dimuat.
- density: $f(S) = (\Sigma\ v_i) \times (1 + V_{utilized} / V_{container})$, yaitu total value yang dikalikan faktor utilisasi volume sehingga konfigurasi yang lebih padat memperoleh nilai lebih tinggi.
- eta_urgency: $f(S) = \Sigma\ v_i \times (1 + 1 / (\max(0, ETA_i) + 1))$, yaitu value setiap paket dikalikan pengali urgensi yang membesar ketika ETA mendekati nol.

Batasan fisik tidak ditangani dengan fungsi penalti, melainkan dengan hanya membangkitkan state yang feasible. Penempatan paket i pada posisi (x, y, z) dengan dimensi (w, d, h) diterima oleh _can_add hanya apabila memenuhi seluruh syarat berikut:

1. Batas peti kemas: $x + w \leq W$, $y + d \leq D$, dan $z + h \leq H$.
2. Kapasitas beban: total berat paket yang sudah terpasang ditambah berat paket i tidak melebihi max_capacity.

3. Tidak saling tumpang tindih: interval paket i tidak beririsan pada ketiga sumbu dengan paket mana pun yang sudah terpasang (fungsi _overlap). Sentuhan pada sisi tidak dihitung sebagai irisan.
4. Penopang (gravitasi): paket i berada di lantai (z = 0), atau terdapat minimal satu paket lain yang permukaan atasnya tepat berada pada ketinggian z dan beririsan dengan alas paket i pada bidang xy dengan luas lebih dari nol.
5. Barang pecah-belah (fragile): tidak ada paket fragile yang menopang paket i secara langsung, dan apabila paket i bersifat fragile maka tidak boleh ada paket yang sudah terpasang tepat di atasnya.

Pada langkah REMOVE, MOVE, dan SWAP, fungsi _all_supported memeriksa ulang bahwa setiap paket yang tersisa pada $z > 0$ masih memiliki penopang dan tidak ditopang oleh paket fragile. Pemeriksaan ini diperlukan karena pengeluaran satu paket dapat membuat paket di atasnya tidak tertopang. Dengan kedua pemeriksaan tersebut, setiap state pada ruang keadaan selalu memenuhi seluruh batasan.

2.3 Implementasi Algoritma Local Search

2.3.1 Hill-Climbing

Implementasi Hill-Climbing terdiri atas kelas dasar HillClimbing yang *inherit* StateSpace, tiga varian turunannya, dan satu varian random restart. Kelas dasar menjalankan kerangka pencarian yang sama untuk seluruh varian, sedangkan aturan pemilihan langkah didefinisikan pada method choose_move di setiap varian. Parameter utamanya adalah max_iterations (default 100), objective_type, dan seed.

Pada setiap iterasi, algoritma membangkitkan seluruh suksesor feasible dari state saat ini melalui neighbors, lalu menghitung $\Delta = f(\text{neighbor}) - f(\text{current})$ untuk setiap suksesor. Seluruh pasangan (Δ, suksesor) diteruskan ke choose_move. Apabila choose_move mengembalikan None, pencarian berhenti dan stopped_at_local_optimum bernilai True. Apabila tidak, state berpindah ke suksesor terpilih dan nilai f diperbarui secara inkremental ($f \leftarrow f + \Delta$) tanpa evaluasi ulang. Langkah dengan $|\Delta| \leq 10^{-9}$ (konstanta EPS) dihitung sebagai sideways move. Pencarian juga berhenti ketika jumlah iterasi mencapai max_iterations. Selama proses, kelas mencatat iterations, neighbors_evaluated, sideways_moves, history_fitness, dan search_duration.

Keempat varian yang diimplementasikan adalah:

- Steepest-ascent: memilih suksesor dengan Δ positif terbesar (Δ > EPS) dan berhenti apabila tidak ada suksesor yang memenuhi.
- Stochastic: memilih satu suksesor secara acak seragam dari seluruh suksesor dengan Δ > EPS dan berhenti apabila tidak ada.
- Sideways: memilih suksesor terbaik seperti steepest-ascent apabila ada Δ > EPS, dan pada kasus tersebut penghitung consecutive_sideways diatur ulang ke nol. Apabila tidak ada langkah naik, algoritma memilih secara acak satu suksesor dengan $|\Delta| \leq$ EPS (plateau) dan menaikkan penghitung. Pencarian berhenti apabila penghitung mencapai max_sideways (default 10) atau tidak ada suksesor datar.
- Random-restart: menjalankan SteepestAscentHillClimbing sebanyak max_restarts (default 10) kali. Setiap restart menggunakan seed acak berbeda yang diambil dari pembangkit bilangan acak utama sehingga initial state-nya berbeda, dan batas max_iterations berlaku untuk setiap restart. State dengan nilai f tertinggi dari seluruh restart dikembalikan. Kelas ini mencatat restarts, iterations_per_restart, fitness_per_restart, dan history_best, serta mengakumulasi iterations dan neighbors_evaluated dari seluruh restart.

2.3.2 Simulated Annealing

Simulated Annealing diimplementasikan pada kelas SimulatedAnnealing yang mewarisi StateSpace dan memakai ruang keadaan serta fungsi objektif yang sama dengan Hill-Climbing. Berbeda dengan Hill-Climbing yang mengevaluasi seluruh suksesor, algoritma ini mengevaluasi satu suksesor acak per iterasi, sehingga biaya per iterasi jauh lebih kecil. Parameter utamanya adalah max_iterations (default 1000), initial_temperature T_0 (default 100,0), cooling_rate α (default 0,995), dan min_temperature (default 0,001).

Pada setiap iterasi, random_neighbor memilih satu suksesor acak, lalu dihitung $\Delta = f(\text{neighbor}) - f(\text{current})$. Aturan penerimaannya adalah:

- $\Delta > 0$: suksesor selalu diterima (accepted_improving).
- $\Delta = 0$: suksesor diterima sebagai sideways move (accepted_sideways).
- $\Delta < 0$: suksesor diterima dengan probabilitas $\exp(\Delta / T)$, yaitu apabila bilangan acak seragam pada [0, 1) lebih kecil daripada probabilitas tersebut (accepted_worsening). Apabila ditolak, state tidak berubah (rejected).

Apabila random_neighbor tidak menemukan suksesor feasible setelah 50 percobaan, iterasi dicatat pada no_neighbor dan state tidak berubah. Setelah setiap iterasi, temperatur

diturunkan secara geometris, T ← α · T. Pencarian berhenti ketika jumlah iterasi mencapai max_iterations atau T < min_temperature. Dengan parameter default, T pada iterasi ke-1000 bernilai sekitar 0,67, sedangkan T baru turun di bawah 0,001 setelah sekitar 2.297 iterasi. Dengan demikian, pencarian default berakhir karena batas iterasi, bukan karena batas temperatur.

Solusi yang dikembalikan adalah state terbaik yang pernah dikunjungi (best), bukan state terakhir, sehingga penerimaan langkah yang menurunkan f tidak mengurangi hasil akhir. Kelas ini mencatat history_current, history_best, history_temperature, history_delta, dan history_acceptance_probability untuk visualisasi. Properti stuck_count (rejected + no_neighbor) dan stuck_frequency (stuck_count dibagi iterations) mengukur seberapa sering pencarian tidak berpindah state.

2.3.3 Genetic Algorithm

Pada permasalahan 3D Container Loading ini, GA diimplementasikan menggunakan pendekatan representasi tak langsung yang dipadukan dengan algoritma *decoder* yang deterministik. Pendekatan ini memastikan bahwa setiap individu (chromosome) yang dievaluasi selalu menghasilkan konfigurasi penempatan paket (state) yang dipastikan valid terhadap seluruh batasan fisik tanpa memerlukan penalti pelanggaran constraint.

Setiap individu dalam populasi direpresentasikan oleh kelas Chromosome yang terdiri atas tiga komponen gen utama untuk n buah paket:

1. order: List[str]: Permutasi dari seluruh ID paket yang menentukan urutan prioritas paket saat dimasukkan ke dalam peti kemas oleh decoder.
2. rotations: Dict[str, int]: Pemetaan setiap ID paket ke indeks orientasi rotasinya. Berdasarkan metode Box.get_rotations(), setiap paket memiliki hingga 6 orientasi dimensi (w, d, h) hasil rotasi 90° dengan duplikasi orientasi simetris yang telah dieliminasi (misalnya kubus hanya memiliki 1 orientasi unik).
3. inclusions: Dict[str, bool]: Variabel biner (boolean) untuk setiap paket yang menentukan apakah paket tersebut diikutsertakan dalam proses pemuatan (True) atau langsung ditempatkan di luar truk (False).

Saat inisialisasi populasi awal (create_random_chromosome), urutan order diacak menggunakan random shuffle, indeks rotations dipilih secara acak merata dari daftar rotasi unik paket, dan inclusions diinisialisasi bernilai True dengan probabilitas 0.85%.

Kemudian, fungsi decode(chromosome) menerjemahkan genotipe (Chromosome) menjadi fenotipe berupa objek State yang valid melalui tahapan berikut:

1. Daftar titik sudut kandidat (candidates) diinisialisasi dengan titik asal sudut kiri-bawah-belakang peti kemas, yaitu [(0, 0, 0)].
2. Algoritma mengiterasi setiap paket sesuai urutan pada chromosome.order. Paket akan dilewati (skip) apabila status inclusions bernilai False atau apabila penambahan berat paket tersebut menyebabkan total muatan melampaui max_capacity truk.
3. Daftar titik kandidat diurutkan secara menaik berdasarkan prioritas koordinat (z, y, x) menggunakan kunci pt: (pt[2], pt[1], pt[0]) agar paket selalu memprioritaskan dasar lantai terendah (z), diikuti bagian paling belakang (y), dan paling kiri (x).
4. Untuk setiap titik kandidat (c_x, c_y, c_z) dan dimensi paket terotasi (w, d, h), titik dinyatakan valid pertama kali apabila memenuhi keempat syarat:
    a. $c_y + d \leq D$, dan $c_z + h \leq H$.
    b. Tidak beririsan (overlap) pada ketiga sumbu dengan paket mana pun yang sudah ditempatkan (PlacedBox.check_overlap).
    c. Berada di lantai dasar ($c_z = 0$) atau memiliki irisan bidang singgung horizontal ($\max(c_x, x_j) < \min(c_x + w, x_j + w_j)$ dan $\max(c_y, y_j) < \min(c_y + d, y_j + d_j)$) dengan permukaan atas paket lain j ($z_j + h_j = c_z$).
    d. Tidak ada satu pun paket penopang tepat di bawahnya ($z_j + h_j = c_z$) yang memiliki atribut is_fragile == True.
5. Jika paket berhasil ditempatkan pada posisi (p_x, p_y, p_z), titik-titik kandidat baru dibangkitkan pada proyeksi sudut paket:
    a. Titik samping kanan $(p_x + w, p_y, p_z)$ dan depan $(p_x, p_y + d, p_z)$.
    b. Titik permukaan atas $(p_x, p_y, p_z + h)$, yang hanya ditambahkan apabila paket tersebut tidak fragile (not box.is_fragile) dan tidak melebihi tinggi truk ($p_z + h < H$).

c. Titik perpotongan proyeksi dengan paket lain yang berada pada elevasi permukaan atas yang sama. Titik kandidat yang berada di luar dimensi kontainer atau tertutup di dalam volume paket yang telah terpasang kemudian dihapus dari daftar

Nilai fitness dari suatu kromosom dihitung menggunakan fungsi evaluate_fitness pada modul objective.py terhadap decoded_state. Terdapat tiga mode fungsi objektif yang didukung (termasuk spesifikasi bonus):

1. Memaksimalkan total value dari seluruh paket yang berhasil dimuat di dalam peti kemas:

$$f_{default}(S) = \sum_{i \in placed} v_i$$

2. Menyeimbangkan total value dengan rasio utilisasi volume peti kemas ($ratio = \frac{V_{utilized}}{W \times D \times H}$) untuk mendorong pengepakan yang lebih padat:

$$f_{density}(S) = \left(\sum_{i \in placed} v_i \right) \times \left(1 + \frac{V_{utilized}}{V_{container}} \right)$$

3. Memberikan bobot pengganda (urgency multiplier) berdasarkan tingkat kedaruratan waktu pengiriman (ETA) agar paket yang harus segera tiba diprioritaskan masuk ke dalam truk:

$$f_{eta}(S) = \sum_{i \in placed} v_i \times \left(1 + \frac{1}{\max(0, ETA_i) + 1} \right)$$

Setelah menghitung fitness function untuk setiap kromosom, dipilih parent untuk dilakukan persilangan yang menghasilkan keturunan baru menggunakan method select_parent. Method select_parent mendukung tiga skema pemilihan induk untuk reproduksi:

1. roulette (default): Probabilitas terpilihnya setiap individu sebanding dengan proporsi nilai fitness-nya terhadap total fitness seluruh populasi ($P(i) = \frac{f_i}{\Sigma f_j}$).
2. tournament: Mengambil sampel acak sebanyak $k = \min(3, N_{pop})$ individu dari populasi, lalu memilih individu dengan nilai fitness tertinggi sebagai induk.

3. rank: Mengurutkan populasi dari fitness terendah ke tertinggi (peringkat 1 hingga n), kemudian melakukan pemilihan proporsional berdasarkan nilai peringkat dengan total bobot $\frac{n(n+1)}{2}$ untuk mencegah dominasi prematur dari satu individu ekstrem.

Kemudian, dilakukan operasi crossover dan mutation untuk individu yang terpilih tersebut.

Crossover Dilakukan dengan probabilitas crossover_rate (default 0.8) menggunakan kombinasi dua teknik:

1. Order Crossover yang memilih dua titik potong acak (cut1 dan cut2), menyalin sub-segmen urutan paket dari parent1 ke child1 pada rentang tersebut, lalu mengisi posisi sisanya secara berurutan menggunakan elemen dari parent2 yang belum muncul agar validitas permutasi tetap terjaga.
2. Uniform Crossover untuk gen rotations dan inclusions yang melakukan hal berikut: Untuk setiap paket, gen orientasi dan inklusi diwariskan secara berpasangan dari parent1 atau parent2 masing-masing dengan peluang (0.5).

Mutasi Dilakukan dengan probabilitas mutation_rate (default 0.2) untuk menjaga keragaman genetik populasi. Saat mutasi terjadi, algoritma memilih secara acak satu dari tiga operator variasi yang berkorespondensi langsung dengan tiga aturan langkah suksesor pada spesifikasi:

1. Menukar posisi dua paket secara acak pada urutan order, serta dengan peluang (0.5) menukar status inclusions antara kedua paket tersebut.
2. Dengan peluang (0.5) memindahkan satu paket dari indeks asal src_idx ke indeks tujuan dst_idx pada urutan order (insertion move), atau dengan peluang (0.5) membalik (flip) nilai boolean inclusions dari suatu paket acak (memasukkan paket luar ke truk atau mengeluarkan paket dari truk).
3. Memilih satu paket secara acak yang memiliki lebih dari 1 orientasi unik (n_rots > 1), lalu menggeser indeks rotasinya secara siklik ke orientasi baru: (cur_rot + randint(1, n_rots - 1)) % n_rots.

Terakhir, Pada setiap generasi hingga mencapai max_iterations (default 100 generasi), dilakukan hal berikut:

1. Sebanyak elitism_count (default 2, dibatasi maksimal floor(Npop/2) individu terbaik dari generasi saat ini langsung disalin tanpa perubahan ke populasi generasi berikutnya (next_population) agar solusi terbaik tidak pernah mengalami degradasi.
2. Selama ukuran next_population belum mencapai population_size, algoritma memilih sepasang induk, melakukan crossover, menerapkan mutate pada kedua anak (offspring), dan mengevaluasi nilai fitness-nya.
3. Di setiap akhir generasi, populasi diurutkan kembali berdasarkan fitness tertinggi. Algoritma memperbarui best_chromosome global dan mencatat riwayat nilai fitness maksimum (history_max_fitness), rata-rata (history_avg_fitness), serta minimum (history_min_fitness), beserta total durasi eksekusi (search_duration) untuk keperluan visualisasi grafik konvergensi dan analisis eksperimen.

BAB III
HASIL DAN ANALISIS EKSPERIMEN

3.1 Setup Eksperimen

Pada eksperimen ini, digunakan 3 dataset untuk menguji hasil dari setiap algoritma yang telah diimplementasikan.

Tabel 3.1.1 Dataset mejiro_express.json

Label Truk	Dimensi (W x D x H)	Kapasitas Maksimum
MejiroExpress_Truck01	10 x 20 x 5	500

No	ID Paket	Dimensi	Value	Weight	Fragile	ETA
1	RusdiPackage	3 x 4 x 1	60	5	False	3
2	McQueenSweets	2 x 3 x 2	85	8	True	1
3	SpecialCarrotCrate	4 x 5 x 2	120	25	False	2
4	GoldShipGadgets	3 x 3 x 2	150	14	True	4
5	TeioShoes	2 x 4 x 1	70	6	False	2
6	RiceShowerBento	2 x 2 x 1	45	4	True	1
7	HokkaidoApples	5 x 6 x 2	180	40	False	5
8	TrainerDocs	2 x 3 x 1	50	3	False	1
9	SpurtDrinkBoxes	4 x 4 x 2	110	22	False	3
10	CrystalTrophy	2 x 2 x 3	200	15	True	6

No	ID Paket	Dimensi	Value	Weight	Fragile	ETA
11	HeavyMachinerySpare	5 x 8 x 3	310	85	False	7
12	WinterBlankets	4 x 6 x 2	95	18	False	4
13	PorcelainTeaSet	3 x 4 x 2	175	12	True	2
14	VeterinarySupplies	3 x 5 x 2	160	20	False	1
15	SteelPlates	4 x 10 x 1	220	90	False	5
16	HokkaidoButter	3 x 3 x 1	65	9	False	3
17	GlassBottles	3 x 4 x 2	130	16	True	2
18	HayBales	6 x 8 x 3	140	50	False	4
19	TrainingBarbells	3 x 6 x 2	190	70	False	5
20	SilkUniforms	3 x 4 x 1	105	7	False	2

Tabel 3.1.2 Dataset multitruck.json

Label Truk	Dimensi (W x D x H)	Kapasitas Maksimum
Truck_A	10 x 20 x 5	300
Truck_B	8 x 15 x 4	250

No	ID Paket	Dimensi	Value	Weight	Fragile	ETA
1	RusdiPackage	3 x 4 x 1	60	5	False	3
2	McQueenSweets	2 x 3 x 2	85	8	True	1
3	SpecialCarrotCrate	4 x 5 x 2	120	25	False	2
4	GoldShipGadgets	3 x 3 x 2	150	14	True	4
5	TeioShoes	2 x 4 x 1	70	6	False	2
6	RiceShowerBento	2 x 2 x 1	45	4	True	1
7	HokkaidoApples	5 x 6 x 2	180	40	False	5
8	TrainerDocs	2 x 3 x 1	50	3	False	1
9	SpurtDrinkBoxes	4 x 4 x 2	110	22	False	3
10	CrystalTrophy	2 x 2 x 3	200	15	True	6
11	HeavyMachinerySpare	5 x 8 x 3	310	85	False	7
12	WinterBlankets	4 x 6 x 2	95	18	False	4
13	PorcelainTeaSet	3 x 4 x 2	175	12	True	2
14	VeterinarySupplies	3 x 5 x 2	160	20	False	1
15	SteelPlates	4 x 10 x 1	220	90	False	5
16	HokkaidoButter	3 x 3 x 1	65	9	False	3
17	GlassBottles	3 x 4 x 2	130	16	True	2

No	ID Paket	Dimensi	Value	Weight	Fragile	ETA
18	HayBales	6 x 8 x 3	140	50	False	4
19	TrainingBarbells	3 x 6 x 2	190	70	False	5
20	SilkUniforms	3 x 4 x 1	105	7	False	2

Tabel 3.1.3 Dataset small.json

Label Truk	Dimensi (W x D x H)	Kapasitas Maksimum
SmallTruck	6 x 8 x 4	150

No	ID Paket	Dimensi	Value	Weight	Fragile	ETA
1	PKG_A	3 x 4 x 2	100	20	False	2
2	PKG_B	2 x 3 x 2	80	15	True	1
3	PKG_C	4 x 4 x 2	150	30	False	3
4	PKG_D	2 x 2 x 1	40	5	False	1
5	PKG_E	3 x 3 x 2	110	18	True	2
6	PKG_F	1 x 5 x 2	90	12	False	4
7	PKG_G	2 x 4 x 1	70	10	False	3
8	PKG_H	3 x 2 x 2	60	8	True	1

3.2 Hasil dan Analisis Hill Climbing

Tabel 3.2.1. Tabel Hasil Hill Climbing dengan dataset *small*

Varian	Mean	Std	Min	Max	Start	Iterasi	Tetangga	Sideways	Local opt	Waktu (s)
Steepest	521,0	27,0	440	530	464	0,7	89	0	10/10	0,001
Sideways (max 25)	530,0	0,0	530	530	464	25,8	1.270	25	10/10	0,013
Stochastic	521,0	27,0	440	530	464	225,5	226	0	10/10	0,006
Random restart (5)	530,0	0,0	530	530	488	2,3	380	0	10/10	0,004
Random restart (10)	530,0	0,0	530	530	488	4,7	763	0	10/10	0,008

Tabel 3.2.2. Tabel Hasil Hill Climbing dengan dataset *mejiro_express*

Varian	Mean	Std	Min	Max	Start	Iterasi	Tetangga	Sideways	Local opt	Waktu (s)
Steepest	2.419,5	87,1	2.300	2.520	2.041	2,0	941	0	10/10	0,020
Sideways (max 25)	2.482,0	67,1	2.350	2.520	2.041	27,4	7.149	25	10/10	0,183
Stochastic	2.111,0	153,9	1.800	2.360	2.041	225,0	225	0	10/10	0,019
Random restart (5)	2.520,0	0,0	2.520	2.520	1.975	9,1	4.151	0	10/10	0,093

Random restart (10)	2.520,0	0,0	2.520	2.520	1.975	18,9	8.537	0	10/10	0,186

Pada dataset small (Tabel 3.2.1), steepest-ascent dan stochastic menghasilkan nilai rata-rata 521,0 dengan simpangan baku 27,0 (minimum 440, maksimum 530). Nilai tersebut konsisten dengan satu dari sepuluh run yang terjebak pada local optimum bernilai 440, sedangkan run lainnya mencapai 530. Steepest-ascent hanya memerlukan rata-rata 0,7 iterasi dan 89 tetangga. Varian sideways dan kedua varian random-restart mencapai 530 pada seluruh seed (simpangan baku 0). Varian sideways membayar hasil tersebut dengan 25,8 iterasi dan 1.270 tetangga, karena rata-rata 25 langkah sideways terpakai pada setiap run. Random-restart dengan 5 restart sudah cukup dan lebih murah, yaitu 380 tetangga dibandingkan 763 tetangga pada 10 restart dengan hasil yang sama.

Perbedaan antarvarian terlihat lebih jelas pada dataset mejiro_express (Tabel 3.2.2). Steepest-ascent berhenti setelah rata-rata 2,0 iterasi pada nilai rata-rata 2.419,5 (simpangan baku 87,1), naik dari nilai awal rata-rata 2.041. Varian sideways meningkatkan rata-rata menjadi 2.482,0 dan menurunkan simpangan baku menjadi 67,1, tetapi biayanya sekitar 7,6 kali lebih besar dalam jumlah tetangga (7.149 dibandingkan 941) dan 0,183 detik dibandingkan 0,020 detik. Varian stochastic paling rendah hasilnya, dengan rata-rata 2.111,0, simpangan baku 153,9, dan nilai maksimum hanya 2.360, sehingga tidak pernah mencapai 2.520 yang diperoleh varian lain. Rata-rata iterasinya mencapai 225,0 dengan jumlah tetangga yang dievaluasi hampir sama (225), sehingga biaya per iterasi sangat kecil, tetapi hasil akhirnya paling rendah. Random-restart dengan 5 maupun 10 restart mencapai 2.520 pada seluruh seed. Dengan 5 restart, biayanya 4.151 tetangga dan 0,093 detik, lebih murah daripada varian sideways yang hasilnya masih lebih rendah. Penambahan restart menjadi 10 tidak meningkatkan hasil dan hanya menggandakan biaya (8.537 tetangga, 0,186 detik).

3.3 Hasil dan Analisis Simulated Annealing

Tabel 3.3.2. Tabel Hasil Simulated Annealing pada dataset *mejiro_express*

Iterasi, T_0, pendinginan	**Mean**	**Std**	**Min**	**Max**	**Stuck freq**	**Trapped freq**	**Worse accepted**	**Waktu (s)**

1000, 100, 0,995	2.296,0	160,5	2.030	2.520	0,849	0,000	19	0,09
3000, 100, 0,998	2.359,0	119,7	2.210	2.520	0,932	0,000	46	0,28
3000, 300, 0,998	2.357,0	115,3	2.210	2.520	0,937	0,000	123	0,22
5000, 300, 0,999	2.427,0	142,1	2.210	2.520	0,951	0,000	235	0,34

Pada dataset mejiro_express (Tabel 3.3), konfigurasi dengan 1.000 iterasi, $T_0 = 100$, dan laju pendinginan 0,995 menghasilkan rata-rata 2.296,0 (simpangan baku 160,5, minimum 2.030). Menaikkan iterasi menjadi 3.000 dengan laju pendinginan 0,998 menaikkan rata-rata menjadi 2.359,0 dan menurunkan simpangan baku menjadi 119,7. Konfigurasi terbaik adalah 5.000 iterasi, $T_0 = 300$, dan laju pendinginan 0,999 dengan rata-rata 2.427,0, tetapi simpangan baku masih 142,1 dan minimum masih 2.210. Pada seluruh konfigurasi, nilai maksimum 2.520 dicapai oleh sebagian seed saja, sehingga tidak ada konfigurasi yang konsisten mencapai nilai tersebut.

Pengaruh temperatur awal terlihat dengan membandingkan konfigurasi kedua dan ketiga, yang berbeda hanya pada T_0 (100 dan 300). Rata-rata nilai akhir hampir sama (2.359,0 dan 2.357,0), sedangkan jumlah langkah memburuk yang diterima naik dari 46 menjadi 123. Dengan demikian, T_0 yang lebih tinggi menambah eksplorasi tetapi tidak meningkatkan hasil pada dataset ini. Peningkatan hasil lebih berkaitan dengan jumlah iterasi dan laju pendinginan yang lebih lambat.

3.4 Hasil dan Analisis Genetic Algorithm

Sesuai dengan spesifikasi tugas besar, eksperimen pada Genetic Algorithm dilakukan dengan menguji pengaruh dua hyperparameter utama, yaitu jumlah populasi dan banyak iterasi/generasi. Parameter lainnya ditetapkan secara konstan sebagai variabel kontrol tambahan berdasarkan nilai default pada implementasi, yaitu crossover_rate = 0.8, mutation_rate = 0.2, elitism_count = 2, metode seleksi roulette, dan fungsi objektif default (total value paket di dalam peti kemas). Eksperimen dibagi menjadi dua skenario utama, di mana setiap konfigurasi parameter dijalankan sebanyak 3 kali (run) untuk mengamati konsistensi hasil stokastik dari algoritma.

Pada skenario pertama, parameter jumlah populasi dijaga konstan (misalnya pada ukuran population_size = 50), kemudian diuji terhadap 3 variasi banyak iterasi, yaitu 50, 100, dan 200 iterasi.

Tabel 3.4.1. Tabel Hasil Eksperimen Skenario 1 dengan dataset mejiro_express.json

Konfigurasi Parameter	**Run ke-**	**Nilai Objective Akhir (Final Value)**	**Durasi Pencarian (detik)**
Populasi = 50, Iterasi = 50	1	2520.0	1.982 s
Populasi = 50, Iterasi = 50	2	2520.0	1.768 s
Populasi = 50, Iterasi = 50	3	2520.0	1.922 s
Rata-rata ± Std	-	**2520.00 ± 0.00**	**1.891 ± 0.110 s**
Populasi = 50, Iterasi = 100	1	2550.0	4.527 s
Populasi = 50, Iterasi = 100	2	2540.0	3.849 s
Populasi = 50, Iterasi = 100	3	2550.0	4.132 s
Rata-rata ± Std	-	**2546.67 ± 5.77**	**4.169 ± 0.341 s**
Populasi = 50, Iterasi = 200	1	2550.0	8.147 s

Konfigurasi Parameter	Run ke-	Nilai Objective Akhir (Final Value)	Durasi Pencarian (detik)
Populasi = 50, Iterasi = 200	2	2550.0	7.934 s
Populasi = 50, Iterasi = 200	3	2550.0	7.627 s
Rata-rata ± Std	-	**2550.00 ± 0.00**	**7.903 ± 0.261 s**

Berdasarkan data empiris pada tabel tersebubt, diperoleh beberapa temuan penting:

1. Pada konfigurasi Iters = 50, ketiga run secara konsisten berhenti pada nilai objektif 2520.0 (rata-rata 2520.00 ± 0.00). Hal ini menunjukkan bahwa 50 generasi belum cukup bagi populasi untuk keluar dari local optimum pertama yang ditemukan oleh decoder.
2. Ketika iterasi ditingkatkan menjadi Iters = 100, algoritma berhasil melewati local optimum 2520.0 dan mencapai nilai 2550.0 pada Run 1 dan Run 3, serta 2540.0 pada Run 2 (rata-rata naik menjadi 2546.67).
3. Pada konfigurasi Iters = 200, seluruh percobaan (Run 1, 2, dan 3) berhasil konvergen secara sempurna pada nilai maksimum 2550.0 (rata-rata 2550.00 $\pm$ 0.00). Hal ini membuktikan bahwa rentang 100–200 iterasi memberikan waktu eksplorasi yang memadai bagi operator order crossover dan mutasi untuk menyusun kombinasi urutan dan rotasi paket yang optimal.

Selain itu, Durasi pencarian berbanding lurus (linier) dengan jumlah iterasi yang dijalankan:

1. Iters = 50 membutuhkan rata-rata 1.891 detik (≈ 0.038 detik/generasi).
2. Iters = 100 membutuhkan rata-rata 4.169 detik ($\approx 2.20 \times$ durasi 50 iterasi).
3. Iters = 200 membutuhkan rata-rata 7.903 detik ($\approx 1.90 \times$ durasi 100 iterasi, atau $\approx 4.18 \times$ durasi 50 iterasi).

Hubungan linier $O(G \cdot N_{pop})$ ini sesuai dengan karakteristik teoretis *Genetic Algorithm*, di mana setiap generasi selalu mengevaluasi sejumlah population_size - elitism_count ($50 - 2 = 48$) individu baru melalui fungsi decode().

Pada skenario kedua, parameter banyak iterasi dijaga konstan sebesar Iters = 100, kemudian diuji terhadap 3 variasi jumlah populasi, yaitu 20, 50, dan 100 individu.

3.5 Analisis Perbandingan dan Evaluasi

Perbandingan antaralgoritma dilakukan pada dataset mejiro_express karena dataset ini yang membedakan kualitas algoritma. Pada dataset small dan multi_truck, hampir seluruh konfigurasi Hill-Climbing dan Simulated Annealing mencapai nilai yang sama (530 dan 1.335), sehingga kedua dataset tersebut tidak dapat digunakan untuk membedakan algoritma. Hasil Genetic Algorithm pada kedua dataset tersebut tidak disertakan dalam perbandingan ini. Tabel 3.5.1 merangkum konfigurasi representatif dari setiap algoritma. Perlu dicatat bahwa Hill-Climbing dan Simulated Annealing dijalankan pada 10 seed, sedangkan Genetic Algorithm dijalankan pada 3 run, sehingga simpangan baku Genetic Algorithm kurang dapat dibandingkan secara langsung.

Tabel 3.5.1. Perbandingan antaralgoritma

Algoritma	Konfigurasi	Mean	Std	Max	Waktu (s)
Hill-Climbing	Steepest	2.419,5	87,1	2.520	0,020
Hill-Climbing	Sideways (max 25)	2.482,0	67,1	2.520	0,183
Hill-Climbing	Stochastic	2.111,0	153,9	2.360	0,019
Hill-Climbing	Random restart (5)	2.520,0	0,0	2.520	0,093
Hill-Climbing	Random restart (10)	2.520,0	0,0	2.520	0,186
Simulated Annealing	1000 iterasi, T_0 = 100, 0,995	2.296,0	160,5	2.520	0,09

Simulated Annealing	5000 iterasi, $T_0 = 300$, 0,999	2.427,0	142,1	2.520	0,34
Genetic Algorithm	Populasi 50, 50 iterasi	2.520,00	0,00	2.520	1,891
Genetic Algorithm	Populasi 50, 100 iterasi	2.546,67	5,77	2.550	4,169
Genetic Algorithm	Populasi 50, 200 iterasi	2.550,00	0,00	2.550	

Dari sisi kualitas solusi, Genetic Algorithm menghasilkan nilai tertinggi, yaitu 2.550 pada seluruh run dengan 200 iterasi dan rata-rata 2.546,67 pada 100 iterasi. Nilai tersebut 30 poin (sekitar 1,2%) di atas 2.520, yang merupakan nilai tertinggi yang dicapai Hill-Climbing dan Simulated Annealing. Hasil ini menunjukkan bahwa 2.520 bukan nilai optimum global pada dataset ini. Hill-Climbing dengan random-restart mencapai 2.520 pada seluruh seed, sedangkan varian Hill-Climbing lainnya berada di bawahnya, dengan urutan sideways (2.482,0), steepest (2.419,5), dan stochastic (2.111,0). Simulated Annealing berada pada rentang 2.296,0 sampai 2.427,0, di bawah random-restart pada seluruh konfigurasi yang diuji.

Dari sisi konsistensi, random-restart Hill-Climbing dan Genetic Algorithm paling stabil. Keduanya memiliki simpangan baku 0,0 pada hampir seluruh konfigurasi (Genetic Algorithm 5,77 pada 100 iterasi). Simulated Annealing paling tidak konsisten, dengan simpangan baku 115,3 sampai 160,5 dan nilai minimum yang masih 2.030 sampai 2.210. Varian stochastic juga tidak konsisten, dengan simpangan baku 153,9.

Dari sisi biaya komputasi, urutan dari yang termurah adalah steepest dan stochastic (sekitar 0,02 detik), random-restart 5 (0,093 detik), Simulated Annealing (0,09 sampai 0,34 detik), dan Genetic Algorithm (1,891 sampai 7,903 detik). Untuk nilai yang sama, yaitu 2.520, Genetic Algorithm dengan 50 iterasi memerlukan waktu sekitar 20 kali lebih lama daripada random-restart 5. Peningkatan dari 2.520 menjadi 2.550 memerlukan waktu sekitar 85 kali lebih lama (7,903 detik dibandingkan 0,093 detik). Selisih ini kemungkinan berasal dari biaya evaluasi per kandidat. Genetic Algorithm harus men-decode setiap kromosom menjadi state dari awal, sedangkan Hill-Climbing mengevaluasi tetangga dari state yang sudah ada. Pada anggaran evaluasi yang sebanding, Simulated Annealing (paling banyak 5.000 evaluasi pada konfigurasi terbesar) tetap lebih rendah hasilnya daripada random-restart

5 (4.151 tetangga). Satu tetangga acak per iterasi dengan stuck frequency 0,85 sampai 0,95 menunjukkan bahwa sebagian besar iterasi tidak menghasilkan perubahan state.

Berdasarkan hal tersebut, tidak ada algoritma yang unggul pada seluruh kriteria. Random-restart Hill-Climbing dengan 5 restart memberikan rasio kualitas terhadap waktu terbaik. Genetic Algorithm dipilih apabila kualitas solusi tertinggi diutamakan dan waktu sekitar 4 sampai 8 detik dapat diterima. Dengan 100 iterasi, Genetic Algorithm sudah mencapai 2.546,67 dengan waktu sekitar separuh dari 200 iterasi. Simulated Annealing pada konfigurasi yang diuji tidak unggul pada kualitas maupun waktu dibandingkan kedua algoritma tersebut. Keunggulan Genetic Algorithm kemungkinan berasal dari rekombinasi urutan penempatan, rotasi, dan inklusi paket yang menjangkau konfigurasi yang sulit dicapai oleh langkah tunggal pada Hill-Climbing dan Simulated Annealing. Dugaan ini belum diuji secara langsung pada eksperimen ini.

BAB IV
KESIMPULAN DAN SARAN

4.1 Kesimpulan

1. Persoalan 3D container loading pada penelitian ini dapat diselesaikan dengan algoritma Hill-Climbing, Simulated Annealing, dan Genetic Algorithm. Ketiganya selalu menghasilkan konfigurasi yang valid terhadap batas peti kemas, kapasitas beban, tumpang tindih, penopang, dan barang pecah-belah, karena batasan tersebut dijaga saat state dibangkitkan.
2. Pada dataset kecil (small dan multi_truck), Hill-Climbing dan Simulated Annealing mencapai nilai yang sama pada hampir seluruh konfigurasi (530 dan 1.335). Perbedaan kualitas algoritma baru terlihat pada dataset mejiro_express dengan 20 paket.
3. Hill-Climbing cepat tetapi bergantung pada state awal. Seluruh run berhenti pada local optimum. Random-restart dengan 5 restart mencapai 2.520 pada seluruh seed dengan waktu 0,093 detik, sedangkan 10 restart tidak meningkatkan hasil dan hanya menggandakan biaya.
4. Simulated Annealing menghasilkan nilai rata-rata 2.296,0 sampai 2.427,0 pada mejiro_express dengan simpangan baku yang besar (115,3 sampai 160,5). Penambahan iterasi dan pendinginan yang lebih lambat meningkatkan hasil,

sedangkan temperatur awal 100 maupun 300 tidak memberikan perbedaan rata-rata yang berarti.

5. Genetic Algorithm menghasilkan solusi terbaik, yaitu 2.550 pada 200 iterasi dan rata-rata 2.546,67 pada 100 iterasi. Algoritma ini satu-satunya yang melampaui 2.520, tetapi memerlukan waktu paling lama (1,891 sampai 7,903 detik).
6. Tidak ada algoritma yang unggul pada seluruh kriteria. Random-restart Hill-Climbing paling efisien, sedangkan Genetic Algorithm paling baik dari sisi kualitas solusi.

4.2 Saran

1. Eksperimen Genetic Algorithm sebaiknya diperluas dengan variasi jumlah populasi, metode seleksi, serta crossover_rate dan mutation_rate. Eksperimen juga sebaiknya dijalankan pada dataset small dan multi_truck dengan jumlah seed yang sama dengan Hill-Climbing dan Simulated Annealing (10 seed) agar perbandingan lebih adil.
2. Pendekatan hibrida dapat dicoba, misalnya menggunakan solusi hasil random-restart Hill-Climbing sebagai bagian dari populasi awal Genetic Algorithm, atau menerapkan Hill-Climbing pada individu terbaik di setiap generasi. Pendekatan ini berpotensi mencapai 2.550 dengan jumlah generasi yang lebih sedikit.
3. Simulated Annealing dapat diperbaiki dengan jumlah iterasi yang lebih besar, pendinginan adaptif atau reheating, serta pemilihan tetangga yang lebih terarah, misalnya dengan memprioritaskan paket bernilai tinggi. Tingginya stuck frequency menunjukkan bahwa sebagian besar usulan langkah terbuang.

PEMBAGIAN TUGAS KELOMPOK

NIM	Nama Anggota	Rincian Tugas & Kontribusi	Persentase Kontribusi
13524027	Aufa Rienaldifaza Ahmad	Pendefinisian dan implementasi *state*, *objective function*, step *constraints, simulated annealing* dan seluruh varian *hill climbing search*, pengerjaan laporan.	33.333…%
13524071	Kalyca Nathania Benedicta Manullang	Implementasi visualisasi 3D state dan dashboard HTML interaktif, pengerjaan fitur bonus multi-truk, penyusunan README, pengujian program, pengerjaan laporan	33.333…%
13524077	Yavie Azka Putra Araly	Implementasi algoritma *genetic algorithm*, Pendefinisian dan implementasi *state* bonus, pengerjaan laporan	33.333…%

REFERENSI

1. Tim Asisten Laboratorium Inteligensi Artifisial. (2026). *Spesifikasi Tugas Besar 1 IF3170 Inteligensi Artifisial 2026/2027*. STEI ITB.
2. Martello, Silvano & Pisinger, David & Vigo, Daniele. (1998). The Three-Dimensional Bin Packing Problem. Operations Research. 48. 256-267. 10.1287/opre.48.2.256.12386.

LAMPIRAN

Lampiran 1: Form Penggunaan Generative AI

Dengan ini, kami:

Nama/NIM : Aufa Rienaldifaza Ahmad / 13524027
Kalyca Nathania Benedicta Manullang / 13524071
Yavie Azka Putra Araly / 13524077

Fakultas/Sekolah : Sekolah Teknik Elektro dan Informatika

Kode Mata Kuliah : IF3170

Nama Mata Kuliah : Intelegensi Artifisial

Judul Tugas : OPTIMASI PENGISIAN PAKET PETI KEMAS MENGGUNAKAN ALGORITMA LOCAL SEARCH

Menyatakan bahwa kami menggunakan AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan	Tidak	0
Pembuatan teks	Tidak	0
Bantuan Diskusi	Ya	2
Pembuatan Gambar	Tidak	0
Debugging	Ya	2

Kami juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini

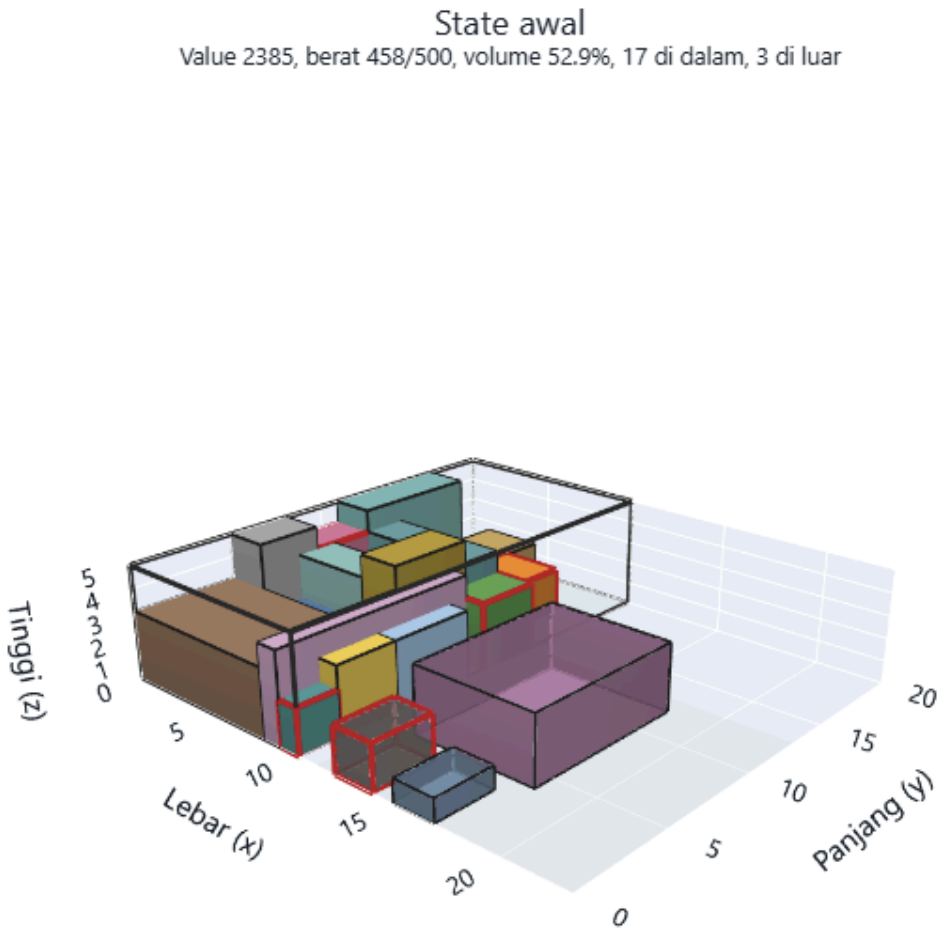
Bandung, 20 Oktober 2026



Aufa Rienaldifaza Ahmad	Kalyca Nathania Benedicta Manullang	Yavie Azka Putra Araly
13524027	13524071	13524077

Lampiran 2: Tangkapan Layar & Visualisasi Program

- **Visualisasi Initial State (3D / GUI):**



- **Visualisasi Final State (3D / GUI):**

